

VoidMC – Greenlight Jury

Framework open source de serveurs Minecraft à grande échelle

TRACK TECHNIQUE – 2026

Adam Brutsaert · Adam Cavillon · Léo Dumont

Le problème

Minecraft est un **vecteur de communication massif**, mais son socle serveur pèse lourd

LES SOLUTIONS DE LA COMMUNAUTÉ

Vanilla

+ Bukkit

+ Spigot

+ Paper

+ Folia

Des couches qui s'empilent sur l'héritage vanilla



Poids inutile

Des fonctionnalités jamais utilisées.



Contrôle limité

Le bas niveau reste masqué.



Performance contrainte

L'héritage vanilla reste coûteux.

→ Notre réponse : un socle **modulable**, taillé pour chaque projet.

Cibles utilisateurs

Qui utilise VoidMC et comment ?



Développeurs de serveurs

Studios et indépendants qui construisent des expériences Minecraft sur mesure

CIBLE PRIMAIRE



Joueurs

Joue sur un serveur construit avec VoidMC via le client Minecraft standard

CIBLE INDIRECTE

Proposition de valeur

Les trois piliers qui différencient VoidMC



Léger

Base vide. Aucune couche
héritée.



Modulaire

Architecture ECS + extensions.



Performant

Rust, Tokio asynchrone, Bevy
ECS.

Positionnement

Comparaison avec l'écosystème existant

Outil	Type	Modularité	Public
Bukkit / Paper	Sur-couche Java	Faible	Extensions de logique de jeu
Folia	Dérivé parallélisé	Moyenne	Gros serveurs
AMP / Pterodactyl	Panneau de gestion	N/A	Hébergeurs
VoidMC	Moteur conçu intégralement	Totale	Studios sur mesure

VoidMC = le **serveur de jeu lui-même**, conçu pour être réécrit pièce par pièce.

Choix de track : Technique

Pourquoi cette orientation et pas une autre



Track Technique

Défi technique central
moteur de serveur complet



Track Solution

Pas de produit
utilisateur final



Entrepreneuriat

Pas de modèle
économique B2C

Verticale : Rust, ECS, parallélisation, protocole binaire, optimisation

Livrable = bibliothèque publique open source sous licence MIT, publiée sur crates.io

Responsabilités

Répartition des domaines au sein de l'équipe



Adam Brutsaert

Cœur

Mesures

Performances



Adam Cavillon

Monde

Logique de jeu



Léo Dumont

Entités

Logique de jeu

Nos objectifs Track Technique

2 obligatoires + 2 choisis pour le PGE

OBLIGATOIRE 1

Évaluer et intégrer de nouvelles technologies

Choix techniques justifiés et intégrés

OBLIGATOIRE 2

Structurer et fiabiliser l'architecture

Structure, CI et couverture mesurée

CHOISI B

Développer les contributions de la communauté

Infrastructure, mobilisation et intégration

CHOISI D

Mesurer et optimiser les performances

Mesures, tests et optimisations vérifiés

Objectif 1 – Mise en pratique

Évaluer et intégrer de nouvelles technologies (veille technologique)



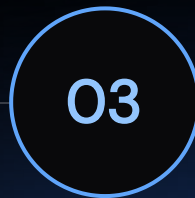
Observer

Documentation et communautés



Comparer

POC reproductibles



Décider

Critères et compromis explicités



Intégrer

Choix appliqués au moteur

Tokio • Bevy ECS • Codec procédural retenus puis intégrés

Objectif 1 – Résultats

Évaluer et intégrer de nouvelles technologies (veille technologique)



Tokio

Écosystème TCP et intégration au
moteur



Bevy ECS

Modularité des systèmes et des
composants



Codec procédural

Moins de répétition, contrôle du
protocole conservé

RÉSULTAT DE LA VEILLE

Trois décisions techniques devenues des composants du moteur

Objectif 1 – Preuves

Évaluer et intégrer de nouvelles technologies (veille technologique)

EXPÉRIMENTATIONS COMPARATIVES

Result Summary

Measured on 2026-06-24.

Runtime	Result	Interpretation
Tokio	18.039 ms	Slower in this tiny timer-heavy POC, but strongest ecosystem fit for VoidMC.
async-std	1.320 ms	Viable alternative with a comfortable API, but less aligned with the rest of our stack.
smol	1.326 ms	Lightweight and fast in this POC, but smaller ecosystem and fewer production examples for our use case.

Decision: keep Tokio. The POC shows that alternatives are technically plausible, but production selection also depends on TCP tooling, documentation, ecosystem support, and integration risk.

Runtime asynchrone : mesures comparatives

Result Summary

Measured on 2026-06-24.

Approach	Final position sum	Interpretation
Bevy ECS	529500.00	Systems/resources compose naturally and mirror VoidMC's plugin model.
Manual loop	529500.00	Works for a prototype, but gameplay features become coupled as behavior grows.

Decision: keep Bevy ECS. The POC demonstrates equivalent behavior while highlighting why the ECS model scales better for modular server features.

Architecture ECS : résultats du POC

SOURCES TECHNIQUES CROISÉES

Async, ECS, and Runtime Architecture

Source	Usage
Tokio Documentation	Async runtime APIs, task model, TCP networking
Tokio Blog	Runtime guidance and performance recommendations
Bevy ECS Documentation	ECS schedulers, components, resources, observers
flume Documentation	Channel behavior and cross-thread communication
tracing Documentation	Structured logging and instrumentation
tracing-flame Documentation	Flamegraph-compatible runtime traces

Runtimes et ECS : variété des sources

Minecraft Protocol and Data

Source	Usage
wiki.org Minecraft Protocol	Protocol concepts and packet structure cross-checking
PaperMC Github	Vanilla-derived source of truth for packet IDs and field layouts
Minecraft generated reports	Registry and data-pack reference material
Local protocol diff: <code>docs/protocol-diff-[1.21.4-to-1.21.5].md</code>	Project-specific watch artifact for version migration

Protocole Minecraft : sources de référence

Objectif 2 – Mise en pratique

Structurer, documenter et fiabiliser l'architecture technique du projet

ARCHITECTURE

- 8 paquets
- 7 bibliothèques publiées
- Responsabilités séparées

INDUSTRIALISATION

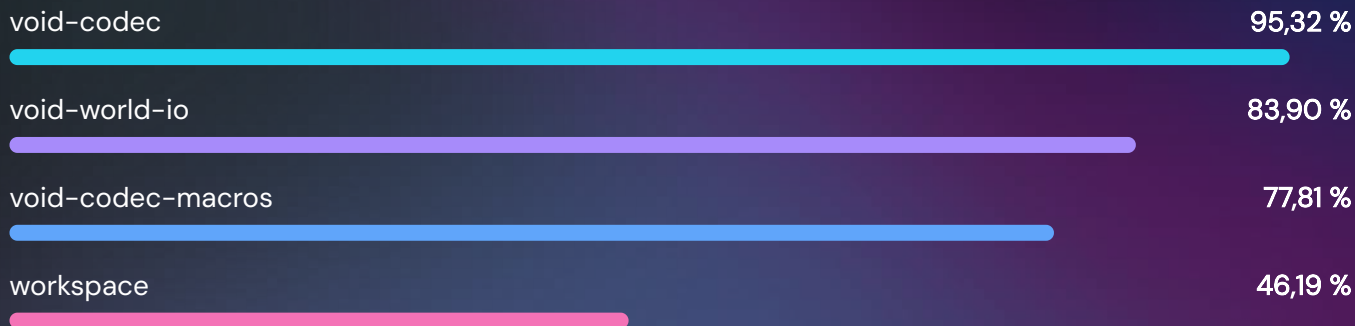
- CI automatisée
- Installation documentée
- Configuration et journaux structurés

Objectif 2 – Résultats

Structurer, documenter et fiabiliser l'architecture technique du projet

COUVERTURE DE TESTS

Mesurer la part du code réellement exercée



Modules critiques couverts, zones faibles identifiées.

Objectif 2 – Preuves

Fiabiliser : couverture de tests et journaux d'exécution

Filename	Function Coverage	Line Coverage	Region Coverage	Branch Coverage
void-codex-macros/src/attrs.rs	100.00% (12/12)	82.99% (122/147)	83.12% (197/237)	- (0/0)
void-codex-macros/src/decode.rs	83.33% (5/6)	73.91% (85/115)	86.23% (119/138)	- (0/0)
void-codex-macros/src/encode.rs	83.33% (5/6)	74.14% (86/116)	85.62% (115/134)	- (0/0)
void-codex-macros/src/lib.rs	50.00% (2/4)	85.71% (12/14)	86.00% (16/20)	- (0/0)
void-codex/src/lib.rs	50.00% (42/83)	89.63% (404/540)	91.59% (827/899)	- (0/0)
void-codex/src/primitives/booleans.rs	100.00% (8/8)	98.25% (56/57)	98.17% (187/189)	- (0/0)
void-codex/src/primitives/containers.rs	100.00% (39/39)	99.37% (314/316)	99.27% (683/688)	- (0/0)
void-codex/src/primitives/fixed_length.rs	100.00% (6/6)	87.93% (51/58)	86.15% (56/65)	- (0/0)
void-codex/src/primitives/floats.rs	100.00% (28/28)	98.28% (114/116)	99.85% (288/288)	- (0/0)
void-codex/src/primitives/integers.rs	100.00% (54/54)	98.32% (362/368)	99.11% (666/672)	- (0/0)
void-codex/src/primitives/uint.rs	100.00% (18/18)	96.58% (113/117)	95.74% (188/198)	- (0/0)
void-codex/src/primitives/void.rs	100.00% (7/7)	98.00% (51/52)	97.00% (99/102)	- (0/0)
void-codex/src/primitives/vars.rs	100.00% (29/29)	96.17% (276/287)	96.42% (431/447)	- (0/0)
void-codex/src/primitives/vari64.rs	100.00% (26/26)	95.78% (289/302)	95.36% (413/433)	- (0/0)
void-data/src/lib.rs	68.00% (32/47)	67.96% (140/206)	67.57% (225/333)	- (0/0)

Couverture des modules critiques Rapport llvm-cov détaillé, exécuté en intégration continue

```
2026-07-16T21:09:42.848107Z INFO voidmc::systems::player: Player connected, player_name: 444lead, player_uuid: 9952f683-ca49-4346-88df-c9c9f16aee42, client_id: 3
  at void/src/systems/player.rs:45 on ThreadId(1)
  in voidmc::network::dispatch_packet with client_id: 3, entity: 131v1
  in voidmc::network::ingest_network_packets

2026-07-16T21:09:42.914861Z WARN voidmc::systems::chunk: Chunk generation throttled, generated_this_tick: 8, max_chunk_generations_per_tick: 8
  at void/src/systems/chunk.rs:172 on ThreadId(1)
  in voidmc::systems::chunk::stream_chunks

2026-07-16T21:09:44.483923Z INFO voidmc::server: Client connection closed, client_ip: 127.0.0.1:58325, error: Custom { kind: UnexpectedEof, error: "Connection closed" }
  at void/src/server.rs:64 on ThreadId(16)
```

Journaux structurés Traces lisibles et exploitables à l'exécution

Objectif 2 – Preuves

Documenter : prise en main et configuration

Prerequisites

- Rust stable toolchain
- Cargo
- Node.js and npm for the documentation site
- A Minecraft client compatible with the protocol version currently targeted by the server

Quick Start

Build all crates:

```
cargo build --workspace
```

Run the example server:

```
cargo run -p voidmc-example
```

The example binds to 127.0.0.1:25565 by default. It creates runtime logs in `logs/`.

Run tests:

```
cargo test --workspace --all-features
```

Configuration Examples

VoidMC configuration is currently code-first through `ServerConfigBuilder`. The example server also reads a small set of environment variables for diagnostics and metrics.

Development Configuration

```
use voidmc::{ServerConfigBuilder, VoidServer};

fn main() {
    let config = ServerConfigBuilder::new()
        .address("127.0.0.1:25565")
        .tick_rate(20)
        .max_players(20)
        .view_distance(8)
        .simulation_distance(8)
        .spawn_chunk_radius(4)
        .initial_chunk_radius(4)
        .motd("VoidMC dev server")
        .metrics_debug(true)
        .metrics_tps_output("logs/tps-dev.csv")
        .build();

    VoidServer::new(config).run();
}
```

Use this shape for local development because it keeps startup fast and enables TPS evidence.

Démarrage rapide Prérequis et lancement en quelques commandes

Configuration Serveur décrit via `ServerConfigBuilder`

Objectif B – Mise en pratique

Développer les contributions de la communauté



Structurer

MIT, CONTRIBUTING, modèles et documentation



Mobiliser

Discussions, issues et échanges communautaires



Intégrer

Retours et contributions externes appliqués

Un dépôt accessible → des échanges actionnables → **des changements intégrés au projet**

Objectif B – Résultats

Développer les contributions de la communauté

2

studios

Évaluent VoidMC sur
des projets réels

12

développeurs

Testent le moteur en
dehors des studios

RETOURS COLLECTÉS



Retour studio

SandBlock Studios et Guill Productions



Retours développeurs

Issus des testeurs externes qualifiés

CONSÉQUENCES SUR LE PROJET



Adaptation de la roadmap

Réordonnée selon les besoins
remontés



Priorisation

De certains points mis en avant



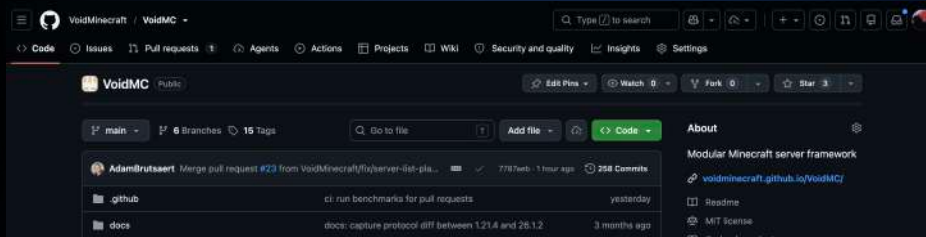
Optimisation ciblée

Sur les points explicitement
mentionnés

Objectif B – Preuves



Développer les contributions de la communauté

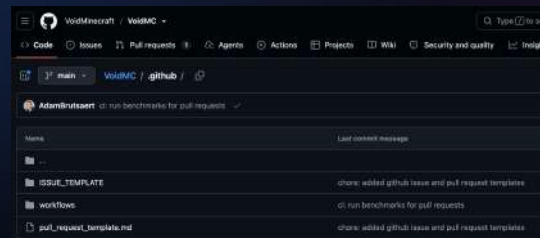


Dépôt public Code, licence et structure accessibles



Guide de contribution

Parcours de contribution explicite



Modèles GitHub Issues et pull requests structurées

Objectif B – Preuves

Retours recueillis auprès de la communauté

RETOUR STUDIO



Mathéo/DiscoowBombie 21:20

On a pu tester Void pour l'un de nos (peut-être) prochains événements courant 2027. Globalement ça tient super bien, on a effectué des tests avec plus de 40 joueurs en simultané. Voici quelques retours :

- L'API développeur est top, peut-être que ça pourrait être plus imagé pour permettre de mieux visualiser les différents rendus
- Il faudrait ajouter une API plus extensive pour manipuler les éléments d'UI en jeu : messages/scoreboard/bossbar, c'est actuellement une version bêta mais c'est ce qu'il manquerait pour rendre nos mini-jeux un peu plus modernes
- Ajouter plus de systèmes pour simplifier la vie aux développeurs, j'ai plein d'idées que je peux te fournir qui nous simplifierait la vie, et on pourrait contribuer pour en ajouter certains (dans la limite de nos ressources disponibles et si on sait que le projet tient entièrement la route)
- L'instrumentation est vraiment pratique, surtout le flamechart qui nous a permis de d'optimiser nos performances un maximum

Globalement c'est un super projet qu'on a hâte d'utiliser, je suis très convaincu de cette bêta et je te fournis un document dans les prochains jours avec les retours des développeurs de nos équipes.

RETOUR DÉVELOPPEUR



Paulem. 20:40

Utiliser Bevy ECS pour un serveur Minecraft en Rust change complètement la réflexion qu'on a par rapport à l'écosystème Spigot ou Paper traditionnel.

Le fait de structurer la logique du jeu en systèmes et composants rend le code ultra-propre, scalable, et bien moins lourd qu'en Java.

L'API est déjà très solide, même s'il reste à compléter l'ensemble des paquets du protocole et des registres d'entités pour s'affranchir totalement des limitations de la beta.

Un projet ultra prometteur ! (modifié)

Objectif D – Mise en pratique

Mesurer, tester et optimiser les performances techniques

UNE DÉMARCHE ITÉRATIVE

🔄 boucle continue



Mesurer

Benchmarks de référence



Identifier

Points chauds et goulots



Optimiser

Correctifs ciblés



Re-mesurer

On recommence

SUR QUATRE AXES



TPS

Stabilité de la boucle



Canaux

Débit des messages



Sérialisation

Coût des conversions



TCP

Trafic réseau

Mesures sur code unitaire, charge et flux.

Objectif D – Résultats

Mesurer, tester et optimiser les performances techniques

19,96 à 19,97

TPS au repos

Serveur au rythme nominal de 20 ticks par seconde

12,857 M/s

Canal flume

Débit des files de messages entre tâches

2,44 à 2,58 μ s

Sérialisation d'un chunk

Chunk de référence d'environ 55 893 octets

0,272 ms

Connexion TCP locale moyenne

64 connexions réussies sur 64, sans assimilation à une latence joueur

Mesures locales, sans extrapolation en débit réseau.

Objectif D – Preuves

Mesurer, tester et optimiser les performances techniques

Result Summary

Measured on 2026-06-24 with cargo bench -p voidmc --bench channel handoff

Benchmark	Result	Throughput
f_lume_unbounded	79.645 us per 1,024 messages	12.857 million messages/s
mutex_vecdeque_prototype	57.881 us per 1,024 messages	17.691 million messages/s

The lock-based prototype is faster in this isolated micro-benchmark. The accepted decision is still flume because the production architecture values explicit ownership, clean thread boundaries, and lower risk of accidental shared-state mutation between Tokio tasks and Bevy ECS systems.

Criterion	Conclusion
Raw micro-benchmark throughput	Mutex prototype wins
Ownership clarity	flume wins
Runtime boundary safety	flume wins
Production fit	flume wins
Decision	Use flume channels for packet, disconnect, and kick routing

Comparatif des canaux



Profil d'exécution (flamechart)

Résultats

Le projet en chiffres

Qualité, écosystème et performances mesurées

46,2 %

couverture globale

codec 95,3 % · persistance
83,9 %

7

bibliothèques publiées

sur crates.io, plus 1 exécutable
serveur

~20 TPS

rythme nominal atteint

19,97 ticks par seconde au
repos

2,44 μ s

sérialisation d'un chunk

chunk de référence d'environ
55 893 octets

Synthèse BTP

Dix scénarios fonctionnels validés

ID	Fonctionnalité	Statut
F1	Network Status	✓
F2	Player Entity Init	✓
F3	Chunk Streaming	✓
F4	Position Sync	✓
F5	Entity Broadcasting	✓

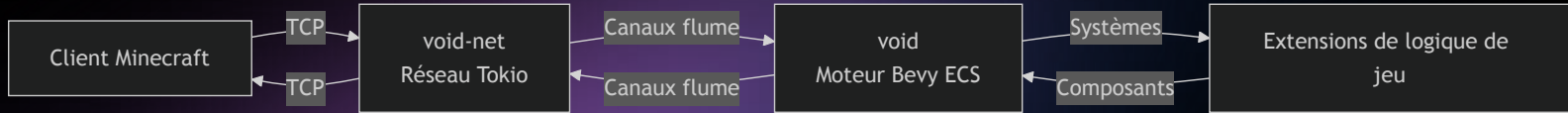
ID	Fonctionnalité	Statut
F6	Chat Event API	✓
F7	World Interaction	✓
F8	World Serialization	✓
F9	Custom Entity Reg	✓
F10	AI Behavior Injection	✓

10 / 10

scénarios fonctionnels validés sur le protocole Minecraft 26.1.2

Architecture exécutée

Architecture à deux fils d'exécution, sans mutex partagé entre réseau et moteur



Séparation réseau et moteur via les canaux flume

Scénario de démonstration

Plan rapide des dix fonctionnalités opérationnelles

CONNEXION

F1

Network Status

MOTD visible

F2

Player Entity Init

Entité ECS créée

F3

Chunk Streaming

Distance 8 chargée

INTERACTION

F4

Position Sync

Mouvement synchronisé

F5

Entity Broadcasting

Deux joueurs visibles

F6

Chat Event API

Message diffusé

F7

World Interaction

Casse et pose

PERSISTENCE ET EXTENSIONS

F8

World Serialization

Conservé après redémarrage

F9

Custom Entity Reg

Entité enregistrée

F10

AI Behavior Injection

Déplacement autonome

Risques et mesures d'atténuation

Identification des risques techniques et mesures d'atténuation



Évolution du protocole Minecraft

Impact : Bloquant

Mesure d'atténuation : codec généré par macro procédurale et différences documentées



Concurrence entre moteur et réseau

Impact : Comportements difficiles à reproduire

Mesure d'atténuation : séparation stricte et canaux flume



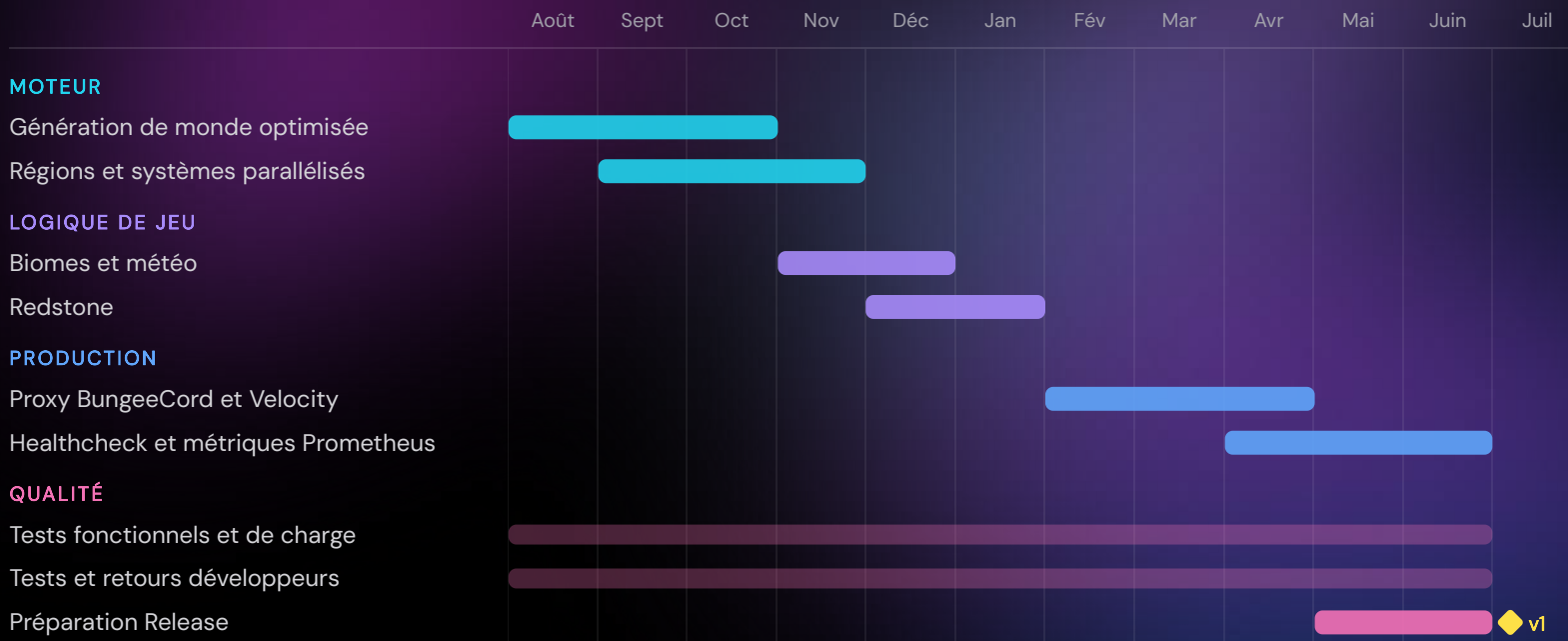
Génération de monde coûteuse

Impact : Dégradation de l'expérience joueur

Mesure d'atténuation : chargement progressif asynchrone

Roadmap vers la v1

D'août 2026 à juillet 2027



Conclusion

- 10/10 scénarios BTP validés
- Architecture modulaire avec qualité mesurée
- Performances instrumentées avec mesures de référence
- SandBlock Studios et Guill Productions comme alpha-testeurs
- Publication de la v1 planifiée pour juillet 2027

Questions ?

Merci

VoidMC - Track Technique

Adam Brutsaert · Adam Cavillon · Léo Dumont